

# Junior Rust Coder Manual: Learning from tor-v3-vanity

---

This manual was extracted from real project chat history and the codebase. It explains **concepts**, **why** things are done a certain way, and **what goes wrong** when they aren't—so you can apply the same ideas in other Rust projects, especially those that use build scripts, `no_std`, or non-standard targets like GPU kernels.

---

## Introduction

**Who this is for:** Developers who know some Rust but haven't dealt with build scripts, `no_std`, custom targets (e.g. NVPTX), or Cargo's linking model. The goal is to make you comfortable reading and changing this project and similar ones.

**What this project does:** It's a Tor v3 vanity `.onion` address generator. It runs a **GPU kernel** (compiled from Rust to NVIDIA PTX) that brute-forces Ed25519 keypairs; the **host** program orchestrates CUDA, loads the kernel, and writes found keys to disk. So you have two "worlds": the host (normal Rust with `std`) and the GPU (Rust with only `core`, different ABI and toolchain).

**How to use this manual:** Read the chapters in order. Each section explains a concept, shows how this project uses it, and describes pitfalls we hit so you know what to look for.

---

## 1. How this project is structured

### 1.1 Two crates, two targets

The repository is a **single Cargo workspace** with two crates:

| Crate         | Path                                               | Runs on           | Purpose                                                                   |
|---------------|----------------------------------------------------|-------------------|---------------------------------------------------------------------------|
| tor-v3-vanity | <code>rust/</code> , <code>rust/src/main.rs</code> | Host<br>(your PC) | CLI, CUDA setup, loading the kernel, receiving results, writing key files |

| Crate                     | Path                                                                                 | Runs on      | Purpose                                                                                 |
|---------------------------|--------------------------------------------------------------------------------------|--------------|-----------------------------------------------------------------------------------------|
| <b>tor-v3-vanity-core</b> | <code>rust/core/</code> , <code>rust/core/src/lib.rs</code> + <code>kernel.rs</code> | GPU (NVIDIA) | Kernel code: derive keypair from seed, compare to prefix, write matching seed to buffer |

The **host** compiles to normal native code (e.g. `x86_64-unknown-linux-gnu`). The **core** crate is compiled **twice**:

1. **For the host** — as an **rlib** so `main.rs` can use its types ( `KernelParams` , `BytePrefix` ) and link with it.
2. **For the GPU** — as a **cdylib** for the `nvptx64-nvidia-cuda` target, which produces a **.ptx** file (NVIDIA's intermediate GPU code). The host does **not** run this code; it passes the **.ptx** to the CUDA driver, which runs it on the GPU.

So: one crate, two builds, two products (host library + GPU kernel).

## 1.2 Data flow (high level)

1. User runs: `t3v --dst mykeys/ myprefix`.
2. **Host** parses args (clap), allocates CUDA buffers, builds a seed and an array of `BytePrefix` (prefix in base32 bytes + mask for partial-byte matching).
3. **Host** compiles the kernel **at build time** (see Chapter 2); at **run time** it loads the **.ptx** via `include_str!(env!("KERNEL_PTX_PATH"))` and passes it to CUDA.
4. **Host** launches the kernel many times: each launch uses a random 32-byte seed; the kernel derives one keypair per thread and checks against the prefixes.
5. **Kernel** writes matching seeds into device buffers and sets a `success` flag.
6. **Host** copies results back, expands the seed to a full key with `ed25519-dalek`, computes the `.onion` address (SHA3 checksum + base32), and writes the Tor secret key file.

Understanding this flow helps when you read `rust/src/main.rs` (host) and `rust/core/src/kernel.rs` (kernel).

## 1.3 Where key code lives

- **Host entry and CUDA loop:** `rust/src/main.rs` — `main()` , `cuda_try_loop()` , `launch!(...)` , result handling.
- **Kernel entry:** `rust/core/src/kernel.rs` — `#[no_mangle] pub extern "ptx-kernel" fn render(params_ptr: *mut KernelParams)`.

- **Shared types (used by both host and kernel):** `rust/core/src/lib.rs` — `KernelParams`, `BytePrefix` (with `DeviceCopy` so they can live on GPU).
  - **Building the kernel:** `rust/build.rs` — runs a second Cargo to produce the `.ptx` and sets `KERNEL_PTX_PATH`.
- 

## 2. Build scripts: what they are and how we use them

### 2.1 What a build script is

A **build script** is a Rust program that Cargo compiles and runs **before** it builds your crate. It lives in `build.rs` at the crate root. It can:

- Run compilers or code generators.
- Emit **directives** to Cargo via `println!`: e.g. `cargo:rustc-link-search=native=/path`, `cargo:rustc-env=VAR=value`, `cargo:rerun-if-changed=file`.
- Cargo then uses those directives when it compiles and links your crate.

So `build.rs` is the place to do “anything that must happen before the rest of the crate can be built.” In our case, we need a **.ptx file** before we can build the host binary, because the host does `include_str!(env!("KERNEL_PTX_PATH"))` — the path must exist at compile time.

### 2.2 Why we run a second Cargo from build.rs

We need to build the **core** crate for the **nvptx64-nvidia-cuda** target with special flags:

- Different **target**: the default build is for the host (e.g. `x86_64`); the kernel must be for `nvptx64-nvidia-cuda`.
- **Rebuild the standard library** for that target: `-Z build-std=core,panic_abort` (the GPU has no prebuilt `std` / `core`; we build a minimal `core` + panic handler).
- **Linker**: we use the **llvm-bitcode-linker** (see Chapter 4), not the host linker.
- **CPU**: `-C target-cpu=sm_75` (NVIDIA compute capability 7.5).

Cargo’s normal “build this workspace” flow doesn’t let you pass different linker/ flavor flags to one crate. So we **invoke Cargo ourselves** from `build.rs` with `cargo rustc -p tor-v3-vanity-core --target nvptx64-nvidia-cuda ... -- -`

`Z ... -C ...`. That produces the `.ptx` in a separate target directory; then we set `KERNEL_PTX_PATH` so the main crate can find it.

## 2.3 The deadlock: why a separate target directory is mandatory

When you run `cargo build`, Cargo takes a **lock** on the project's **target** directory (e.g. `target/`). It holds that lock until the whole build (including the build script) finishes.

Our build script runs **another** `cargo` command. If that inner Cargo uses the **same** target directory, it will try to acquire the **same** lock. But the outer Cargo already holds it and won't release it until the build script exits. The build script won't exit until the inner Cargo finishes. So both wait on each other **forever** — the build appears “stuck” (sometimes for hours) with no extra output.

**Fix:** Give the kernel build its **own** target directory so it uses a **different** lock:

```
CARGO_TARGET_DIR=target/nvptx-kernel
```

In our `build.rs` we set `kernel_target_dir = target_dir.join("nvptx-kernel")` and pass it as `CARGO_TARGET_DIR` when we run the inner `cargo rustc`. Then the `.ptx` is under `target/nvptx-kernel/nvptx64-nvidia-cuda/release/deps/...ptx`.

**Lesson:** Any time your build script runs Cargo (or any tool that might lock the same directory), use a **separate** `CARGO_TARGET_DIR` for that sub-build.

## 2.4 What the kernel build actually runs

From `build.rs`, the inner command looks like:

```
cargo rustc -p tor-v3-vanity-core --target nvptx64-nvidia-cuda \
-Z build-std=core,panic_abort --release -- \
-Z unstable-options \
-C link-self-contained=+linker \
-C linker-flavor=llbc \
-C target-cpu=sm_75
```

- **-Z build-std=core,panic\_abort** — Build the `core` library and the panic runtime for the NVPTX target from source. This is required because there's no prebuilt `std / core` for the GPU. It also implies **panic = abort** (no unwinding) for that build.

- `link-self-contained=+linker` — Use the **bundled** linker from the `llvm-bitcode-linker` component instead of an external binary.
- `linker-flavor=llbc` — Use the LLVM bitcode linker protocol (produces PTX). The default for `nvptx64-nvidia-cuda` is a different “linker” (`rust-ptx-linker`) with a different CLI; we avoid that by switching flavor.
- `target-cpu=sm_75` — NVIDIA GPU architecture (compute capability 7.5).

## 2.5 Why “Cargo.lock does not exist” / “unable to build with the standard library”

Building `core` for NVPTX with `-Z build-std=core,panic_abort` requires the **source** of the standard library. Rustup installs that as the **rust-src** component. If that component isn’t installed, the compiler fails with something like:

```
"/lib/rustlib/src/rust/library/Cargo.lock" does not exist,
unable to build with the standard library, try:
rustup component add rust-src --toolchain nightly-x86_64-unknown-linux-gnu
```

So the **one-time setup** for this project includes:

```
rustup component add llvm-bitcode-linker llvm-tools rust-src --toolchain nightly
```

We deliberately **do not** reimplement a “pre-flight” check in `build.rs` that parses `rustup component list` to verify `rust-src`. In the past that led to bugs: we derived the toolchain from the `CARGO` executable path (which can be the rustup proxy, not the nightly path), or we required the word “installed” in the output even though `rustup component list --installed` doesn’t print that word, so the check always failed. Letting the **kernel build** fail and show the compiler’s own message is more reliable.

## 3. no\_std and the GPU: one crate, two builds

### 3.1 What no\_std means

Normally a Rust crate uses the **standard library** (`std`): heap allocation, I/O, threads, etc. `no_std` means the crate does **not** link to `std`. You only have the **core** library (types like `Option`, `Result`, basic traits, no heap, no I/O). That’s what runs on embedded devices or, in our case, on a GPU: there is no OS, no malloc, no files.

The **kernel** runs on the GPU, so it must be `no_std`. It can only use crates that support `no_std` (e.g. `byteorder` with `default-features = false`, `ed25519-compact`).

### 3.2 Conditional `no_std` in this project

The **core** crate is built **for the host** (so the host can use `KernelParams`, `BytePrefix`) **and** for the GPU. On the host we want normal `std`; on the GPU we want `no_std`. So we make `no_std` **conditional**:

```
#![cfg_attr(target_arch = "nvptx64", no_std)]
```

So: when the target is **nvptx64** (GPU), we enable `no_std`; otherwise we don't. The **kernel** module is also only compiled for the GPU:

```
#[cfg(target_arch = "nvptx64")]  
mod kernel;
```

So on the host, `core` is a normal crate with `std` and no `kernel`; on the GPU it's `no_std` and includes `kernel`.

### 3.3 Panic handler and the “duplicate lang item” error

In a `no_std` environment, there is no default way to **panic**. You must provide a **panic handler** with `#[panic_handler]`. In `kernel.rs` we have:

```
#[panic_handler]  
fn panic(_: &::core::panic::PanicInfo) -> ! {  
    use core::arch::nvptx::*;  
    unsafe { trap() }  
}
```

That's correct **for the GPU build**. But if the **same** crate were built for the **host** with `no_std` **and** this panic handler, and then linked into a binary that uses `std`, you'd get:

```
duplicate lang item in crate `tor_v3_vanity_core`: `panic_impl`  
  first definition in `std`  
  second definition in `tor_v3_vanity_core`
```

The **lang item** `panic_impl` is the function the compiler calls when a panic happens. `std` provides one; our crate would provide another. The linker sees two definitions and fails.

**Fix:** Use **conditional** `no_std` and **only** compile the panic handler for the GPU. We already have `#![cfg_attr(target_arch = "nvptx64", no_std)]`; the `#[panic_handler]` is inside `kernel.rs`, which is only compiled when `target_arch = "nvptx64"`. So on the host we never define a panic handler, and there's no duplicate.

**Lesson:** If a crate is built for both a `std` target and a `no_std` target, make sure the **panic handler** exists only in the `no_std` build (e.g. behind `#![cfg(target_arch = "nvptx64")]`).

### 3.4 Unstable features for PTX: `abi_ptx` and `stdarch_nvptx`

The GPU kernel uses **PTX intrinsics** from `core::arch::nvptx` (e.g. `_block_dim_x()`, `_thread_idx_x()`, `trap()`). Those are **unstable**. We enable them with:

```
#![cfg_attr(target_arch = "nvptx64", feature(abi_ptx, stdarch_nvptx))]
```

- **abi\_ptx** — PTX calling convention / ABI.
- **stdarch\_nvptx** — The `core::arch::nvptx` module and its intrinsics.

Older nightlies used a feature named **stdsimd**; that was **removed**. If you see `unknown feature 'stdsimd'`, switch to **stdarch\_nvptx**. Feature names in nightly can change; check the compiler's help message.

### 3.5 `panic = "abort"` and where it can be set

For `no_std` with `-Z build-std=core,panic_abort`, we need **panic = abort** (no unwinding). In Cargo, **panic** is a **profile** setting, **not** a per-package setting. So you cannot write in `Cargo.toml`:

```
[package]
panic = "abort"    # Invalid: panic may not be specified in a package profile
```

You must set it in a **profile**:

```
[profile.release]
panic = "abort"
```

```
[profile.dev]
panic = "abort"
```

In our project we **don't** set that at the workspace level because the **host** build can use the default (unwind). Only the **kernel** build uses `panic=abort`, and that's forced by `-Z build-std=core,panic_abort` in the `cargo rustc` invocation in `build.rs`. So we avoid the duplicate panic handler by keeping the host build normal and the kernel build isolated in that separate invocation.

---

## 4. Nightly and the PTX linker

### 4.1 Why nightly is required

The `nvptx64-nvidia-cuda` target and the features we use (`abi_ptx`, `stdarch_nvptx`) are **unstable**. They are only available on the **nightly** compiler. So the project must be built with `cargo +nightly build ...`. There is no stable path for NVPTX yet.

### 4.2 The official path: `llvm-bitcode-linker`

The Rust project provides an **llvm-bitcode-linker** as a nightly component. It takes LLVM bitcode (what `rustc` produces for the NVPTX target) and produces a `.ptx` file. So we don't need the old third-party **ptx-linker** (which often breaks with new Rust/LLVM and has a different CLI).

We enable the right behavior by passing to the **kernel** build:

- `-C link-self-contained=+linker` — Use the linker from the “self-contained” set (the `llvm-bitcode-linker`).
- `-C linker-flavor=llbc` — Use the “llbc” (LLVM bitcode) linker protocol.

Those are passed **after** `--` in the `cargo rustc` call so they apply to that crate's compilation/link step.

### 4.3 Why `-C linker=/path` didn't fix “rust-ptx-linker not found”

The `nvptx64-nvidia-cuda` target is defined (in the compiler) with a default **linker** named **rust-ptx-linker**. So `rustc` literally tries to run a binary with that name. Passing `-C linker=/path/to/llvm-bitcode-linker` in `RUSTFLAGS` does **not** change that name for this target in some toolchains — `rustc` still looks for a program called **rust-ptx-linker** and fails with “linker rust-ptx-linker not found”.



The robust fix is to **change the linker protocol** with **linker-flavor=llbc** (and **link-self-contained=+linker**) so that the **bundled** llvm-bitcode-linker is used instead of invoking a separate binary by name. We do that in the `cargo rustc` arguments in `build.rs`, so we never rely on a `rust-ptx-linker` binary.

## 4.4 crate-type: cdylib and rlib

In `rust/core/Cargo.toml` we have:

```
[lib]
crate-type = ["cdylib", "rlib"]
```

- **rlib** — A Rust library for linking into other Rust code. The **host** needs this so it can use `KernelParams`, `BytePrefix`, etc.
- **cdylib** — A C-compatible dynamic library. For the **nvptx64** target, the compiler/linker produce **PTX** (the “dynamic library” is the `.ptx` module). So the GPU build produces the `.ptx` we load at runtime.

We also have in `core/Cargo.toml`:

```
[build]
target = "nvptx64-nvidia-cuda"
```

so that when Cargo builds the **core** crate for the **default** target (host), it still knows the “library” can be built for another target in a separate step. The actual kernel build is done by our `build.rs`, not by Cargo’s normal dependency build.

---

## 5. Cargo and Rust concepts that caused real bugs

### 5.1 Traits must be in scope

We use **clap** with the derive API:

```
#[derive(clap::Parser)]
struct Args { ... }
let args = Args::parse();
```

The **parse()** method comes from the **Parser** trait. Derive macros generate an impl of **Parser** for **Args**, but they do **not** put the trait into scope. So you must write:

```
use clap::Parser;
```

otherwise you get: “**no function or associated item named `parse` found for struct `Args`**”. This is a common mistake with derive-based APIs (serde, clap, etc.): the **trait** that provides the method must be in scope where you call it.

## 5.2 Edition 2021 prelude

In **Rust 2021**, **TryInto** / **TryFrom** are in the **prelude**, so you don’t need `use core::convert::TryInto` in `no_std` code. Leaving that import in triggers a “redundant import” or “unused” warning. Removing it keeps the kernel crate clean.

## 5.3 cfg for the NVPTX target

The only valid **target architecture** for PTX in Rust is **nvptx64** (64-bit PTX). There is no **nvptx** (32-bit) in practice. So we use `target_arch = "nvptx64"` only. Using `target_arch = "nvptx"` can cause **unexpected\_cfgs** warnings and confusion.

## 5.4 cargo: directives and when the build script reruns

In `build.rs` we emit:

- `cargo:rerun-if-changed=core` — If anything under `core/` changes, Cargo will rerun the build script on the next build.
- `cargo:rerun-if-env-changed=KERNEL_PTX_PATH` — If that env var changes, rerun.
- `cargo:rustc-env=KERNEL_PTX_PATH=...` — Set the env var for the **main** crate’s compile (so `env!("KERNEL_PTX_PATH")` in `main.rs` gets the path to the .ptx).

So the flow is: build script runs → builds kernel → finds .ptx → sets `KERNEL_PTX_PATH` → Cargo compiles `main.rs` with that env var set.

---

# 6. CUDA and linking

## 6.1 Driver vs toolkit

- **NVIDIA driver** — Provides **libcuda.so** at **runtime**. It talks to the GPU. You might have this (e.g. from a distro package or runfile) but **not** the full toolkit.

- **CUDA toolkit** — Headers and libraries needed to **build** programs that use CUDA (e.g. **libcudart**, **libcublas**). Often installed separately (e.g. `/usr/local/cuda`), or distro packages that put libs in `/usr/lib/x86_64-linux-gnu`).

So “CUDA not found” at **build** time usually means: the **toolkit** (or at least the libs) isn’t installed or isn’t on the search path. **nvidia-smi** only confirms the driver; it doesn’t guarantee you can **link** against CUDA.

## 6.2 Configurable CUDA path in build.rs

Different machines put CUDA in different places. So in `build.rs` we have `cuda_lib_dir()` that:

- Checks **TARGET** (for cross-compilation we may need a different arch’s libs).
- Uses **AARCH64\_CUDA\_LIB\_DIR** or **X86\_64\_CUDA\_LIB\_DIR** when building for those targets (e.g. cross-compiling from arm64 to x86\_64 or vice versa).
- Otherwise uses **CUDA\_LIB\_DIR** (exact directory containing e.g. `libcuda.so`), or **CUDA\_HOME / CUDA\_PATH** (we append `/lib64`).
- Defaults to `/usr/local/cuda/lib64` and adds a **fallback** path for distro-installed libs: `/usr/lib/x86_64-linux-gnu` or `/usr/lib/aarch64-linux-gnu`.

We then emit `cargo:rustc-link-search=native=<dir>` so the linker looks there for `-lcuda`, `-lcudart`, etc.

## 6.3 Cross-compilation: linker and CUDA libs

When building the **host** binary for another architecture (e.g. **aarch64** on an x86\_64 host, or **x86\_64** on an aarch64 host), two things are needed:

1. **Linker** — A GCC (or similar) that produces binaries for the **target** arch. Cargo uses the **CARGO\_TARGET\_LINKER** env var (e.g. **CARGO\_TARGET\_AARCH64\_UNKNOWN\_LINUX\_GNU\_LINKER=aarch64-linux-gnu-gcc**). The Makefile sets this and uses `$(firstword ...)` so only one path is passed (we had a bug where two paths were concatenated with a space and the linker failed).
2. **CUDA libs for the target** — On an x86\_64 host, the installed CUDA libs are x86\_64. To build an **aarch64** binary you need **aarch64** CUDA libs (e.g. from an extracted ARM64 toolkit or from a Jetson). So we use **AARCH64\_CUDA\_LIB\_DIR** (or **X86\_64\_CUDA\_LIB\_DIR** when building x86\_64 on an aarch64 host) and pass that into `build.rs` so the right directory is used for **rustc-link-search**.

The **PTX kernel** is always the same (nvptx64); only the **host** binary's architecture changes when you cross-compile.

---

## 7. Reproducible builds and the Makefile

### 7.1 One setup command

We document a single one-liner so anyone can get a working build:

```
rustup install nightly && rustup target add nvptx64-nvidia-cuda --toolchain nightly && rustup c
```

After that, **cargo +nightly build --release** (or **make**) should be enough. If something is missing, the **kernel** build fails with a clear compiler message (e.g. add rust-src).

### 7.2 Why we removed the pre-flight check from build.rs

We used to run a “pre-flight” in `build.rs` that called **rustup component list** and **rustup target list** and parsed the output to see if rust-src and the nvptx target were installed. That caused **false “missing rust-src”** errors even after **make setup** succeeded. Causes:

- **Toolchain detection** — We derived the toolchain from the **CARGO** env var. When you run **cargo +nightly build**, CARGO is often the **rustup proxy** (`~/ .cargo/bin/cargo`), not a path inside `~/ .rustup/toolchains/nightly-....`. So we sometimes queried the wrong toolchain or failed to detect “nightly.”
- **Output parsing** — We required the word “installed” in the line, but **rustup component list --installed** prints lines like **rust-src-x86\_64-unknown-linux-gnu** with no word “installed,” so the check always failed.

**Lesson:** Don't reimplement rustup's or the compiler's checks by parsing their output; formats change and the logic is easy to get wrong. Prefer to run the real build and let the compiler emit its own error and fix suggestion.

### 7.3 Makefile: portable paths and idempotent setup

- **DEB\_DIR** and similar are set with `?=` and **\$(CURDIR)** so paths are **repo-relative** and work on any machine.

- The **setup** target runs the rustup install/target/component commands. Those are **idempotent** (safe to run multiple times). Making **amd64** / **arm64** depend on **setup** ensures deps are installed before build.
- **check-cuda** looks for **libcuda.so** in the configured (or fallback) path and fails with a clear message (and optional **make install-cuda** hint on Debian/Ubuntu) so users fix link errors before a long build.

## 8. Troubleshooting reference

| Symptom                                                                      | Cause                                                                                                 | What to do                                                                                                                                                                                                 |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cargo.lock does not exist / unable to build with the standard library</b> | Missing <b>rust-src</b> for nightly                                                                   | Run the one-time setup; <b>rustup component add rust-src --toolchain nightly</b>                                                                                                                           |
| Build <b>stuck</b> at <b>tor-v3-vanity(build)</b>                            | Either first kernel build is slow (many minutes) or <b>deadlock</b> (inner Cargo locking same target) | Ensure <b>CARGO_TARGET_DIR</b> for the kernel build is <b>separate</b> (e.g. <b>target/nvptx-kernel</b> ). Run with <b>-vv</b> to see progress                                                             |
| <b>linker rust-ptx-linker not found</b>                                      | Target expects that binary; we use llvm-bitcode-linker                                                | Don't install ptx-linker. Use <b>linker-flavor=llbc</b> and <b>link-self-contained=+linker</b> in the kernel build; ensure <b>llvm-bitcode-linker</b> (and <b>rust-src</b> ) are installed                 |
| <b>unknown feature stdsimd</b>                                               | Feature renamed in newer nightly                                                                      | Use <b>stdarch_nvptx</b> for PTX intrinsics                                                                                                                                                                |
| <b>duplicate lang item panic_impl</b>                                        | Crate defines panic handler and is also linked with <b>std</b>                                        | Make <b>no_std</b> conditional ( <b>cfg_attr(target_arch = "nvptx64", no_std)</b> ) and define the panic handler only for the GPU (e.g. in <b>kernel.rs</b> under <b>#[cfg(target_arch = "nvptx64")]</b> ) |
|                                                                              |                                                                                                       | Add <b>use clap::Parser;</b>                                                                                                                                                                               |

| Symptom                                                          | Cause                                                   | What to do                                                                                                                                                                                            |
|------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| no function or associated item named parse found for struct Args | Parser trait not in scope                               |                                                                                                                                                                                                       |
| panic may not be specified in a package profile                  | Cargo doesn't allow <b>panic</b> per-package            | Set <b>panic = "abort"</b> in <b>**[profile.release] / ** [profile.dev]</b> at the root if needed; for our kernel we pass <b>-Z build-std=core,panic_abort</b> in <b>build.rs</b> instead             |
| unable to find library -lcuda / -lcudart                         | CUDA libs not on default path                           | Set <b>CUDA_LIB_DIR</b> or <b>CUDA_HOME</b> ; or install the toolkit ( <b>make install-cuda</b> on Debian/Ubuntu). Check with <b>ls</b> that <b>libcuda.so</b> exists in the dir you pass             |
| Pre-flight "missing rust-src" even after <b>make setup</b>       | Bug in custom check (wrong toolchain or output parsing) | Prefer <b>no</b> pre-flight; let the kernel build fail and show the compiler message. If you keep a check, use toolchain <b>nightly</b> explicitly and check <b>rustup</b> exit status before parsing |

## 9. Summary: lessons to take away

1. **Build script sub-Cargo** — If `build.rs` runs Cargo (or anything that locks the same directory), use a **separate CARGO\_TARGET\_DIR** to avoid deadlock.
2. **PTX** — Use **nightly**, **llvm-bitcode-linker**, and **linker-flavor=llbc**; don't depend on the legacy **ptx-linker** or parsing its CLI.
3. **no\_std + host** — When one crate is built for both host and GPU, use **conditional no\_std** and a **panic handler only for the GPU** to avoid duplicate **panic\_impl**.
4. **Traits in scope** — Derive macros don't import traits; put **use clap::Parser** (and similar) where you call the methods.
5. **Don't reimplement toolchain checks** — Let the compiler/rustup fail with their own messages; parsing their output is fragile.

6. **CUDA** — Distinguish **driver** (runtime) vs **toolkit** (build); make the lib path **configurable** (env vars + fallbacks) and document why “CUDA missing” can happen.
  7. **Reproducibility** — One setup command, idempotent setup, repo-relative paths; avoid brittle pre-flight checks in build scripts.
- 

*This manual was generated from project agent transcripts and the tor-v3-vanity codebase. For build and usage details, see the main [README](#) and [CLAUDE.md](#).*